AI-ENABLED SMART EVENT REGISTRATION & TICKETING MANAGEMENT SYSTEM

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Course: DBE & DSD | Course Code: 25CS1302E

Submitted by

2510030272 – Sharanya

2510030277 – Akhila Goud

2510030283 – Nandini

2510030286 – Akshara

Under the guidance of

Dr. Rafeeq Mohammed



Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

Academic Year 2026–2027

ABSTRACT

Events such as campus programmers, workshops, seminars, cultural festivals and public gatherings require an organized way to publish event information, collect registrations, allocate tickets, record payments and verify participants at the venue. When these activities are handled through separate forms, spreadsheets, messages and manual ticket checks, organizers may face duplicated records, unclear ticket availability, slow entry and limited information about expected attendance.

This report presents the design of an AI-Enabled Smart Event Registration and Ticketing Management System. The proposed platform connects event discovery, user registration, ticket reservation, payment-status recording, QR-based e-ticket verification and organizer reporting through a common application workflow. The project also proposes AI features that can support event recommendations and attendance-demand forecasting. These features are intended to help participants discover relevant events and help organizers plan resources using available registration and historical attendance information.

The Review-1 material establishes the problem, societal need, objectives, feasibility and novelty. The literature-review material compares common database-backed ticketing approaches and identifies gaps in integration, real-time status, analytics and verification. Review-3 documents the proposed methodology, data preparation, system architecture, technology options, implementation stages and testing strategy. Review-4 provides the project objectives, intended implementation stack, functional-result checklist, sample test cases and illustrative performance values. Any sample values in Review-4 are explicitly treated here as illustrative and must be replaced with measurements from the actual application before final submission.

The report follows the supplied project-document template and organizes the material in a sequence that incorporates the four review stages. It describes the proposed architecture, database entities, workflows, AI design, testing plan, risks, limitations and future enhancements. The system's final claims about implementation status, accuracy and response time should be supported by screenshots, logs and repeatable tests collected by the project team.

LIST OF FIGURES

Figure 1.1 Problem context and stakeholder view

Figure 2.1 End-to-end event registration workflow

Figure 2.2 Proposed layered architecture

Figure 2.3 User-to-event database relationship

Figure 3.1 Data collection and preprocessing pipeline

Figure 3.2 AI recommendation workflow

Figure 3.3 Demand forecasting workflow

Figure 3.4 QR ticket verification workflow

Figure 4.1 Development lifecycle

Figure 4.2 Deployment view

Figure 5.1 Functional testing flow

Figure 5.2 Performance measurement procedure

Figure 6.1 Proposed future mobile extension

LIST OF TABLES

Table 1.1 Stakeholders and their needs

Table 1.2 Project objectives and indicators

Table 2.1 Existing approach comparison

Table 2.2 Gap analysis

Table 2.3 Functional requirements

Table 2.4 Non-functional requirements

Table 2.5 Database entities

Table 2.6 Technology stack

Table 3.1 Candidate AI methods

Table 3.2 Data fields and purposes

Table 4.1 Implementation modules

Table 4.2 API endpoint plan

Table 5.1 Functional test cases

Table 5.2 Performance metrics

Table 5.3 Risks and mitigations

Table 6.1 Future enhancements

TABLE OF CONTENTS

Abstract

List of Figures

List of Tables

1. INTRODUCTION

1.1 Background and Context

1.2 Problem Statement

1.3 Need and Societal Relevance

1.4 Objectives

1.5 Scope and Stakeholders

1.6 Novelty and Contributions

2. METHODOLOGY AND SYSTEM DESIGN

2.1 Review-1 Feasibility

2.2 Literature Review Approach

2.3 Existing Systems and Gap Analysis

2.4 Requirements Specification

2.5 Proposed Workflow

2.6 System Architecture

2.7 Database Design

2.8 Security and Privacy

3. AI METHODOLOGY

3.1 Data Sources

3.2 Data Preprocessing

3.3 Recommendation Module

3.4 Demand Forecasting

3.5 Fraud and Anomaly Detection
3.6 Model Evaluation
4. SYSTEM IMPLEMENTATION PLAN
4.1 Development Environment
4.2 Frontend
4.3 Backend and APIs
4.4 Database Implementation
4.5 Ticket and QR Module
4.6 Payment and Notification Integration
4.7 Admin Dashboard
4.8 Deployment
5. EXPERIMENTS, TESTING AND RESULTS
5.1 Testing Strategy
5.2 Functional Test Cases
5.3 Integration and Security Testing
5.4 Performance Analysis
5.5 Results and Discussion
5.6 Limitations
6. CONCLUSION AND FUTURE WORK
References
Appendix A: Review Mapping
Appendix B: Evidence to Attach
Appendix C: Sample Data Dictionary

AI-Enabled Smart Event Registration & Ticketing Management System

1. INTRODUCTION

Events are important for academic learning, professional networking, cultural participation and community engagement. Colleges and organisations conduct seminars, technical workshops, hackathons, club activities, conferences and celebrations throughout the year. These events usually involve several connected tasks: publishing event details, collecting participant information, managing capacity, issuing tickets, tracking payment status and confirming attendance at the venue. The quality of this process affects both the participant experience and the organiser's ability to run an event efficiently.

A manual process can be sufficient for a small gathering, but it becomes difficult to maintain as the number of events and registrations increases. Participant details may be stored in multiple spreadsheets; ticket confirmation may be sent through separate messaging channels; payment status may need to be checked manually; and venue staff may have to search lists during entry. Such fragmentation increases the possibility of mistakes and delays. It also makes it harder to estimate how many registered participants will actually attend.

The proposed AI-Enabled Smart Event Registration and Ticketing Management System is designed as a centralized web-based platform. It brings the key ticketing steps into a connected workflow and proposes AI-assisted functions to improve event discovery and resource planning. The project is not only a ticket-booking page: it considers the relationships among users, events, registrations, tickets, payment records and verification outcomes.

The report consolidates the team's Review-1 project definition, literature-review work, Review-3 methodology and implementation plan, and Review-4 design and sample results. The technology names are preserved from the reviews where relevant. Review-3 considered React.js with a Python backend and MySQL/MongoDB options, while Review-4 specifies React.js, Node.js with Express, MongoDB, REST APIs and a Python scikit-learn AI module. For implementation, the team should select and consistently document one final stack rather than mix alternative stacks in the deployed application.

1.1 Background and Context

Digital registration tools make it possible for participants to sign up without visiting an organizer in person. A ticketing platform extends this by tracking the number and type of tickets, linking a booking to the participant, and recording whether the ticket is valid. A QR code can encode a ticket identifier or signed verification token, allowing authorized entry staff to validate the ticket with a scanner or camera-enabled device.

AI can add a decision-support layer when sufficient data is available. A recommendation model can rank events using a participant's stated interests, selected categories and prior interactions. A forecasting model can estimate likely registrations or attendance using historical event patterns and current booking pace. These predictions should be treated as estimates, not guarantees, and the organiser should remain able to review the information before making operational decisions.

1.2 Problem Statement

The problem addressed by this project is the lack of a single, integrated and intelligent workflow for event discovery, participant registration, ticket reservation, payment-status tracking and entry verification. When these activities are handled by separate tools, event organizers may struggle to maintain consistent records, prevent duplicate registrations, monitor ticket availability and estimate attendance. Participants may experience slow booking, unclear confirmation and delays at entry.

The proposed system aims to reduce fragmentation by connecting these functions through a shared application and database. AI-assisted recommendations and demand forecasting are proposed to improve relevance and planning. The platform should provide clear booking status and verifiable tickets while preserving access controls for users and administrators.

1.3 Need for the Project and Societal Relevance

A unified registration and ticketing process can benefit educational institutions, event clubs, training providers, conference organizers and participants. Organizers gain a consistent view of event capacity, registrations and ticket status. Participants can view event information, reserve tickets and retrieve their ticket from one interface. Venue staff can verify tickets using a defined workflow rather than searching long paper lists.

The societal value is mainly operational: reducing avoidable queues, improving access to event information, lowering administrative effort and helping organizers plan resources. Demand estimates can support decisions about seating, refreshments, volunteers and entry gates. The system should not claim that AI guarantees attendance or eliminates fraud; its value depends on data quality, implementation quality and human oversight.

Stakeholder	Need	Expected benefit
Attendee	Find relevant events and book easily	Clear event details, booking status and ticket access
Event organizer	Manage events, capacity and registrations	Centralized records and basic reports
Entry staff	Verify tickets quickly	QR validation and clear valid/invalid outcomes
Institution/club	Coordinate multiple events	Consistent workflow and reusable records
System administrator	Control access and maintain data	Role-based permissions, backups and audit records

1.4 Project Objectives

1. Develop an easy-to-use web platform for event discovery, registration and ticket booking.
2. Maintain linked records for users, events, registrations, tickets and payment status.
3. Generate a unique e-ticket identifier and support QR-based ticket validation at event entry.
4. Propose AI-based event recommendations using participant interests and available interaction history.
5. Propose attendance-demand forecasting and organiser reports to support capacity and resource planning.
6. Test core workflows systematically and document measured outcomes rather than relying on illustrative values.

1.5 Scope of the Project

The core scope includes attendee registration and login, event browsing and filtering, event creation and management by an authorised organiser, ticket reservation, booking confirmation, QR ticket generation, ticket validation and an organiser dashboard. Payment integration is included as a planned API integration; during development, a sandbox or mock payment flow may be used until a verified gateway integration is configured.

AI functionality is scoped to recommendation and demand forecasting as described in Review-4. Fraud/anomaly detection and natural-language processing are considered candidate extensions from Review-3. These extensions should only be presented as implemented if the team has built and tested them. The report distinguishes proposed capabilities from verified implementation evidence.

1.6 Novelty and Contributions

The project's contribution is the integration of several commonly separated functions into a single workflow. The proposed design links participant identity, event capacity, registration, ticket allocation, payment status and entry verification. AI features are positioned as supporting services within this workflow rather than as disconnected demonstrations. The organiser dashboard is intended to connect operational ticketing records with basic sales and attendance summaries.

The novelty is therefore practical integration and decision support rather than the invention of a new machine-learning algorithm. The project should be evaluated on correctness, usability, security, consistency of ticket status and the usefulness of AI outputs.

2. METHODOLOGY AND SYSTEM DESIGN

2.1 Review-1 Feasibility Analysis

Review-1 asks the team to explain the problem, societal need, objectives, proposed solution, feasibility and novelty. The system is feasible as a software project because its main components can be developed using common web-development tools and a database. A laptop or desktop is sufficient for development and demonstration. A camera-enabled phone or webcam can be used to scan QR tickets; specialised sensors are not mandatory for the basic system.

Category	Proposed requirement	Feasibility note
Hardware	Laptop/desktop with internet access	Sufficient for development and local demonstration
Optional entry device	Phone camera or webcam	Can scan QR tickets; no dedicated sensor required
Frontend	React.js, HTML5, CSS3, JavaScript	Supports responsive event and booking interfaces
Backend	Node.js and Express	REST endpoints, validation and business logic
AI service	Python with scikit-learn	Separate recommendation/forecasting logic
Database	MongoDB (Review-4 stack)	Stores users, events, bookings and tickets
Developer tools	VS Code, Git, Postman	Coding, version control and API testing
External services	Payment and email/SMS APIs	Use sandbox/test credentials during development

Dataset requirement: the transactional application does not require a public dataset to perform basic registration and ticketing. AI training or evaluation does require relevant historical or synthetic event data. If the team uses synthetic records, that fact must be clearly stated; synthetic data may demonstrate a workflow but does not establish real-world predictive accuracy.

Ticketing Management System 2.2 Literature Review Approach

The literature-review presentation lists search terms such as event registration management system, online event ticketing system, event management database system, QR-based event ticketing system and ticket reservation management system. It identifies Google Scholar, IEEE Xplore and the ACM Digital Library as places to search for academic material. The comparison in the presentation is a high-level comparison of approaches using MySQL, MongoDB, PostgreSQL, web applications and QR verification.

For the final academic submission, the team should attach complete bibliographic details for any papers actually consulted. The comparison below reproduces the conceptual categories from the review presentation; it should not be treated as a verified systematic review of five individually cited publications unless the team adds the corresponding sources.

Approach described in review	Strength noted	Limitation noted	Improvement direction
MySQL-based registration	Simple relational design	Limited advanced ticket management	Integrate ticket lifecycle
MongoDB event information	Flexible document storage	Relationships can be harder to manage	Define consistent linked records
PostgreSQL online booking	Strong transaction support	Real-time features may be limited in basic designs	Refresh availability promptly
MySQL with web ticket booking	Straightforward reservation	Limited analytics	Add organiser reports
Database plus QR verification	Faster ticket validation	Verification may be separate	Integrate QR into the booking lifecycle

2.3 Gap Analysis

The project review identifies four broad gaps. First, a feature gap occurs when registration, ticketing and verification are implemented as separate functions. Second, a data gap occurs when user, event, registration and ticket relationships are not consistently represented. Third, a real-time gap occurs when ticket availability and booking status do not update promptly. Fourth, an integration gap occurs when payment and verification are added as independent tools.

The proposed improvement is to connect the main entities and workflow through a central application. A successful booking should reserve available capacity, create a registration record, issue a ticket after the defined payment/confirmation rule, and make the ticket status available to the verification service. The design must also address failure cases, such as a payment that is pending, a repeated payment callback or a QR code scanned twice.

Gap	Risk if unaddressed	Proposed response
Fragmented workflow	Staff repeat work across tools	Connect registration, ticketing and verification
Inconsistent records	Incorrect ownership or ticket status	Use stable IDs and validation rules
Stale availability	Overselling or failed bookings	Check and update capacity atomically
Separate payment flow	Unclear payment/booking state	Store payment status and process callbacks safely
Limited analytics	Poor resource planning	Provide counts and cautious forecasts

2.4 Functional Requirements

ID	Requirement	Priority
FR-01	A user can create an account and sign in.	High
FR-02	A user can browse events and filter by category, date, city or price where fields exist.	High
FR-03	An authorised admin can create, update and delete event records.	High
FR-04	The system checks ticket availability before confirming a booking.	High
FR-05	The system stores a booking linked to the user and event.	High
FR-06	A confirmed ticket has a unique identifier and QR representation.	High
FR-07	An authorised scanner or entry user can validate ticket status.	High
FR-08	The system records payment status or a test-mode payment result.	High
FR-09	The system displays a basic organiser report.	Medium
FR-10	The system can display AI-generated event recommendations when the module is available.	Medium
FR-11	The system can display a demand estimate when sufficient data is available.	Medium
FR-12	The system provides clear errors for invalid input and unavailable tickets.	High

2.5 Non-Functional Requirements

Quality	Requirement and rationale
Usability	Forms should use clear labels, input validation and understandable messages.
Performance	Measure API and database response time under representative test conditions.
Reliability	Avoid duplicate ticket issuance and maintain consistent booking states.
Security	Use password hashing, authentication, authorisation and server-side validation.
Privacy	Collect only needed personal information and restrict access to it.
Maintainability	Separate frontend, API, database and AI responsibilities.
Scalability	Design APIs and queries so the system can be moved to a hosted environment.
Accessibility	Use readable contrast, keyboard-friendly controls and responsive layouts.
Auditability	Record important admin actions and ticket-validation events where appropriate.

2.6 Proposed End-to-End Workflow

7. Admin creates an event and defines date, venue, ticket capacity and ticket types.
8. Attendee opens the portal, registers or signs in, and browses the event catalogue.
9. Attendee selects an event and ticket quantity; the backend validates the request and available capacity.
10. The system creates a pending booking or reservation according to the chosen booking policy.
11. Payment is completed through a sandbox/production gateway or a clearly labelled test-mode process.
12. After the required confirmation rule is satisfied, the system stores the final booking status and issues a unique ticket.
13. The ticket is displayed to the attendee as a QR e-ticket and may be sent through an enabled notification channel.
14. At the venue, the verification endpoint checks that the ticket exists, belongs to the expected event and has not already been used.
15. The system records entry status and updates organiser reports; the AI service may use suitable aggregated historical records for recommendations or forecasts.

The workflow should be implemented with explicit state transitions. For example, a booking can move from pending to confirmed, cancelled or failed. A ticket can move from issued to used or revoked. A payment callback must be authenticated and handled idempotently so repeated callbacks do not create multiple tickets.

2.7 System Architecture

The Review-4 architecture uses React.js for the frontend, Node.js and Express for the backend, MongoDB for persistent data, REST APIs for application communication, payment and QR services for ticketing, and Python with scikit-learn for AI. This report adopts that stack as the target implementation architecture because it is the most specific stack stated in Review-4. Review-3 mentioned alternative combinations such as Flask/Django and MySQL; those are retained as design alternatives only and should not be mixed into the final implementation description unless the code actually uses them.

Layer	Component	Responsibilities
Presentation	React.js UI	Event catalogue, login, booking, ticket display, admin dashboard
Application	Node.js + Express	Authentication, event CRUD, booking rules, API validation
Persistence	MongoDB	Users, events, registrations/bookings, tickets and payment records
AI service	Python + scikit-learn	Recommendations and demand estimation
External integration	Payment, email/SMS, QR	Payment status, notification delivery and QR rendering/validation
Development/testing	VS Code, Git, Postman	Implementation, version control and API verification

Architecture flow: Attendee/Admin → React UI → Express REST API → MongoDB; the API can call the Python AI service for recommendations or forecasting and can communicate with payment/notification services. Responses return through the API to the UI. QR validation calls a protected endpoint that checks the stored ticket state.

2.8 Database Design

The literature-review presentation proposes the connected entities User, Event, Registration, Ticket and Payment. The Review-4 implementation material names MongoDB collections for users, events, tickets and bookings. The final schema may use a booking collection to represent registration, but the meaning of each record should be consistent throughout the application.

Entity/collection	Example fields	Purpose
users	_id, name, email, phone, passwordHash, role, preferences	Account identity, role and preference data
events	_id, name, description, category, dateTime, venue, capacity, availableCount, price	Event catalogue and capacity
bookings	_id, userId, eventId, quantity, status, createdAt	Connects a participant to an event and reservation state
tickets	_id, bookingId, eventId, userId, qrToken, status, issuedAt, usedAt	Individual ticket identity and entry state
payments	_id, bookingId, providerRef, amount, status, updatedAt	Payment transaction state and reference
attendanceLogs	_id, ticketId, eventId, scannedAt, result, scannedBy	Optional record of verification attempts

Relationships: one user can create many bookings; one event can have many bookings; a booking can contain one or more tickets; a booking can have one or more payment attempts depending on policy; and a ticket can have multiple scan attempts in the audit log but should only be accepted for entry according to the event's single-use rule.

2.9 Data Integrity and Ticket State

The most important integrity rule is that the system must not confirm more tickets than the event can accommodate. Capacity should be checked on the server rather than trusting the browser. If two users request the final available ticket at the same time, the backend must use an atomic update, transaction, or equivalent concurrency-safe strategy so only one request succeeds.

Ticket state must also be unambiguous. A valid, unused ticket for the correct event can be accepted. A ticket that does not exist, has been revoked, belongs to a different event or has already been used should be rejected with a clear result. The verification operation should update the used state in a way that prevents two entry gates from accepting the same ticket simultaneously.

2.10 Security and Privacy Considerations

- Store password hashes, never plaintext passwords.
- Use authentication tokens securely and check roles on protected admin routes.
- Validate and sanitise all input on the server; do not rely only on frontend validation.
- Keep payment secrets and database credentials in environment variables, not source control.
- Use provider-side verification for payment callbacks and reject untrusted client claims.
- Do not place sensitive personal details directly inside a QR code; use a random or signed token that is checked server-side.
- Limit personal-data collection and restrict access to attendee records.
- Use HTTPS in hosted deployments, backups and appropriate log retention.
- Provide a clear process for revoking a ticket or correcting a booking.

3. AI METHODOLOGY

3.1 Data Sources and Collection

Review-3 identifies registration forms, payment gateway logs, event web/mobile interactions, QR or RFID check-in records, online surveys, public/ticketing APIs and historical event datasets as possible sources. Not every source is required for a working prototype. The core booking application can operate from user-provided details, event metadata and booking records. A model should only be trained on fields that the team has permission to collect and that are relevant to the intended prediction.

Data source	Possible fields	Use
Registration form	Event ID, category, selected preferences, booking timestamp	Registration workflow and recommendations
Event catalogue	Category, venue/city, date, capacity, price	Search, filtering and similarity
Booking records	Quantity, lead time, booking status	Demand trends and basic analytics
Check-in records	Ticket ID, scan timestamp, accepted/rejected result	Attendance estimation and validation monitoring
Payment status	Status, amount, provider reference	Booking confirmation logic; not raw card data
Feedback	Rating or short text, if collected	Optional event analysis/NLP

Data collection should be transparent. The team should avoid collecting sensitive information unless a clear requirement and suitable safeguards exist. Payment card details should remain with the payment provider rather than being stored by the project database.

3.2 Data Preprocessing

The Review-3 plan includes removing duplicate registrations, standardising names and contact formats, filtering spam or bot submissions, handling missing values and extracting predictive attributes. The implementation should use repeatable preprocessing steps so training and prediction apply the same transformations.

16. Validate required fields such as event identifier, event date, ticket capacity and booking status.
17. Remove exact duplicate records where the duplicate can be identified safely; preserve valid separate bookings.
18. Normalise categories, city labels and status values into consistent forms.
19. Parse timestamps into a consistent time zone and derive booking lead time.
20. Flag incomplete payment records for review rather than automatically treating them as successful.
21. Handle missing optional values with a documented strategy; do not invent values for critical transaction fields.
22. Encode categorical variables and scale numeric variables only where the chosen model requires it.
23. Split historical data into training and test sets without allowing future information to leak into training.
24. Document the dataset size, source, date range, class balance and any synthetic data generation.

A critical distinction is between correcting malformed data and changing the meaning of valid records. For example, two bookings with the same name are not necessarily duplicates. De-duplication should use appropriate identifiers and business rules, not name similarity alone.

3.3 AI Event Recommendation Module

The Review-4 objectives propose personalised event suggestions based on user interests and booking history. A simple initial implementation can use content-based ranking: represent each event by category, keywords, venue/city and other non-sensitive attributes, then rank events according to the participant's stated preferences. This approach can work before the system has a large number of users.

Collaborative filtering is listed in Review-3 as another candidate. It uses patterns of interactions among users and events to recommend items that similar users have engaged with. It generally requires sufficient interaction data and careful handling of cold-start cases. For a new user, the interface can request categories of interest or show popular upcoming events rather than pretending that a personalised history exists.

3.3.1 Recommendation workflow

25. Read the user's chosen interests and permitted interaction history.
26. Filter out events that are cancelled, past, full or otherwise unavailable.
27. Calculate a relevance score using matching categories, keywords or historical interactions.
28. Rank eligible events and return a small set with understandable reasons where possible.
29. Collect optional feedback such as clicks or saves to improve future ranking.
30. Evaluate relevance using a labelled test set or a documented user-feedback procedure.

The recommendation module should not guarantee that a user will like an event. It should make suggestions that are relevant to the information available and provide ordinary search/filter controls so participants remain in control.

3.4 Attendance-Demand Forecasting

The project objectives include estimating demand so organisers can prepare seating, refreshments, security and other resources. A forecast may use historical registrations and attendance, event category, day of week, time until the event, current booking pace and cancellation/no-show rates. The prediction target must be defined precisely: expected registrations and expected actual attendees are different quantities.

A basic baseline is to report current confirmed registrations and a historical attendance ratio. If sufficient labelled history is available, a regression model can estimate attendance count, while a classification model can estimate whether an individual booking is likely to result in attendance. The team should begin with a simple baseline and compare a more complex model against it.

Candidate method	Possible target	Evaluation
Historical mean / ratio baseline	Expected attendance count	MAE or RMSE
Linear regression / tree regressor	Attendance count	MAE, RMSE, residual inspection
Random Forest regressor	Attendance count with nonlinear patterns	MAE/RMSE on held-out events
Random Forest classifier	Individual no-show risk, if labels exist	Precision, recall, F1, ROC-AUC where appropriate

Forecasts should include the date/time of calculation and, if possible, an uncertainty range. Organisers should be told that the estimate is based on available records and may change as new registrations and cancellations arrive.

3.5 Fraud and Anomaly Detection

Review-3 lists Random Forest/XGBoost for no-show or suspicious-booking patterns and Isolation Forest for unusual booking spikes. These are candidate methods rather than proof that fraudulent activity can be identified accurately. A useful first layer is deterministic validation: prevent duplicate ticket IDs, enforce booking limits where appropriate, verify payment status and reject already-used tickets.

An anomaly detector can flag patterns such as unusually rapid booking volume, repeated failed attempts or a high number of bookings sharing a device/network signal. Such signals are imperfect and can affect legitimate users, so they should normally trigger review or additional verification rather than automatic accusations. The system should record why a booking was flagged and allow an authorised administrator to review it.

3.6 Natural-Language Processing Extension

Review-3 also mentions NLP text classification to categorise attendee support queries and feedback. This feature is optional and should only be included if the team has a suitable set of labelled examples. A prototype could classify feedback into categories such as booking issue, payment issue, venue query or general suggestion. The output should be presented as a suggested category, not as a guaranteed understanding of the participant's meaning.

3.7 AI Evaluation and Responsible Use

The evaluation must match the task. For recommendations, the team can assess precision at K, click-through rate or relevance ratings from a small, clearly described evaluation. For attendance forecasting, MAE or RMSE compares predicted and actual counts. For fraud or no-show classification, precision and recall should be reported together because a model can appear accurate by predicting the majority class. The data split should be event-aware where possible to test performance on events not used during training.

The project should not claim a model accuracy, fraud-detection rate or recommendation quality unless the team has run the experiment and retained its evidence. If there is no historical dataset, describe the AI modules as prototypes or proposed methods and use synthetic examples only to demonstrate processing.

4. SYSTEM IMPLEMENTATION PLAN

4.1 Development Environment

Review-4 names VS Code, Git and Postman as development tools. React.js is the frontend, Node.js and Express implement the backend API, MongoDB stores application records, and Python with scikit-learn is used for AI functionality. The team should keep a README containing installation steps, environment variables, database setup and test commands so that another team member can reproduce the demonstration.

Component	Selected target technology	Role
Frontend	React.js, HTML5, CSS3, JavaScript	Responsive user and admin interface
Backend	Node.js + Express	REST API and business rules
Database	MongoDB	Persistent user, event, booking, ticket and payment records
AI	Python + scikit-learn	Recommendation and forecasting prototype
API testing	Postman	Request/response and error-path tests
Version control	Git and GitHub	Change history and collaboration
QR	QR-generation/scanning library	Render and read ticket token
Payment	Provider sandbox API or labelled mock mode	Test payment lifecycle without real charges

4.2 Frontend Implementation

The React interface should be organised into reusable pages and components. Attendees need a home or event-list page, event detail view, sign-up/login forms, booking summary, ticket page and profile/booking history. Organisers need a protected dashboard, event creation/edit form, attendee list and summary reports. The user interface should show loading, success, empty and error states rather than leaving the user uncertain when an API call fails.

Event search should apply filters consistently with the fields stored in the database. Booking controls should display availability returned by the server but must not be treated as the final authority on capacity. The server validates the ticket quantity at the time the booking request is processed.

4.3 Backend and REST API Design

The Express backend acts as the trusted layer between the browser, database and integrated services. It should validate input, enforce access control, apply booking rules and return predictable JSON responses. The exact endpoint names can be adjusted to match the implementation; the following table is a design plan, not a claim that every endpoint already exists.

Method and route (example)	Purpose	Access
POST /api/auth/register	Create an account after validation	Public
POST /api/auth/login	Authenticate user	Public
GET /api/events	List/search events	Public
GET /api/events/:id	Read event details	Public
POST /api/events	Create an event	Admin
PATCH /api/events/:id	Update an event	Admin
DELETE /api/events/:id	Remove/cancel an event	Admin
POST /api/bookings	Request a ticket reservation	Authenticated user
GET /api/bookings/me	List current user's bookings	Authenticated user
GET /api/tickets/:id/qr	Retrieve/render a ticket QR	Ticket owner/authorised role
POST /api/tickets/validate	Validate a QR token at entry	Authorised entry staff
GET /api/admin/reports	View ticket and attendance summaries	Admin
GET /api/ai/recommendations	Retrieve ranked events	Authenticated or preference-aware
GET /api/ai/demand/:eventId	Retrieve demand estimate	Admin/organiser

4.4 Database Implementation

MongoDB documents should use stable identifiers and consistent field names. A booking should reference the user and event by ID rather than copying all event information as the only source of truth. Some event details may be snapshotted into the booking for audit purposes, but the design should distinguish the current event record from the details that were valid when the booking was made.

The database should enforce unique constraints where supported, especially for email addresses and ticket tokens. Indexes can be considered for event date, category, booking owner, event ID and ticket status as the dataset grows. Indexes should be selected from actual query patterns and measured rather than added without reason.

4.5 Ticket Generation and QR Validation

Each issued ticket should have a unique, hard-to-guess identifier or signed token. The QR code should contain the token or a URL/token combination that can be checked by the server. It should not expose the user's phone number, email address or other personal information. A scanner sends the token to a protected validation endpoint, which checks existence, event association, status and whether the ticket has already been used.

31. Create the ticket only after the booking meets the confirmation policy.
32. Generate a unique token and store its association with the ticket record.
33. Render the QR code in the ticket page and optionally include a human-readable booking reference.
34. At entry, scan the QR and submit the token to the server.
35. Return a clear accepted, already-used, invalid, revoked or wrong-event result.
36. Record the scan outcome and update the ticket state atomically when accepted.

4.6 Payment and Notification Integration

The payment integration should follow the selected provider's documented checkout and callback process. The application should not trust a browser message that says payment succeeded. It should verify the provider response or signed callback, compare the expected amount and booking reference, and update the payment state in an idempotent way. Test credentials should be used for demonstration until production approval is available.

Email or SMS notifications can send booking confirmation, ticket access, reminders and cancellation notices. Notification failure should not silently erase a valid booking; the ticket should remain available in the account so the participant can retrieve it. The system should log delivery status without storing unnecessary message contents.

4.7 Admin Dashboard and Reports

The organiser dashboard can summarise the number of events, confirmed bookings, issued tickets, completed payments and verified entries. It can provide event-wise filtering and exportable reports if that feature is implemented. Attendance should be calculated from accepted entry records rather than inferred solely from the number of tickets issued.

Demand forecasting should be shown separately from actual counts. A chart should label whether it displays confirmed registrations, predicted attendance or verified check-ins. This avoids confusing a model estimate with a measured outcome.

4.8 Deployment and Maintenance

The prototype can first be run locally for development and demonstration. A hosted deployment can place the frontend, API and AI service in separate managed environments or containers, with a managed database. Deployment must include environment configuration, HTTPS, restricted database access, logging, backups and a process for restoring data.

Maintenance activities include dependency updates, error monitoring, database backups, reviewing failed payment callbacks, removing obsolete test records and monitoring AI performance. A model should be retrained only when there is enough new, relevant and appropriately governed data. Model versions and training-data dates should be recorded so the team can understand changes in prediction behaviour.

5. EXPERIMENTS, TESTING AND RESULTS

5.1 Testing Strategy

Testing should cover individual functions, interactions among modules, end-to-end booking and verification, security controls and performance under representative load. Review-4 includes a sample table of functional tests and illustrative performance values. The functional table below is a test plan based on that review. The status must be updated with actual outcomes from the team's current build.

Test level	What to verify	Evidence
Unit	Input validation, ticket token generation, state transitions	Automated test output or function-level results
API	Request schemas, response codes, permissions and errors	Postman collection/export
Integration	Frontend, API, database, QR and payment status flow	Screenshots and server/database logs
End-to-end	User registers, books, receives ticket and validates entry	Recorded walkthrough or screenshots
Security	Unauthorised admin access, invalid token, duplicate scan	Negative-test results
Performance	Response time and error rate at a stated load	Postman/DevTools measurements

5.2 Functional Test Cases

The following cases preserve the functions listed in Review-4. Expected outcomes are defined so the team can run the tests and record the actual result. Do not mark a test as Pass unless it was executed against the application.

ID	Test case	Expected result	Actual result/status
TC-01	Register a new user	Account created after valid input	Fill after execution
TC-02	Log in with correct credentials	Authenticated session/token returned	Fill after execution
TC-03	Log in with incorrect password	Request rejected with safe message	Fill after execution
TC-04	Admin adds an event	Event saved and visible in list	Fill after execution
TC-05	Admin updates event details	Updated values returned and displayed	Fill after execution
TC-06	Unauthorised user attempts admin action	Access denied	Fill after execution
TC-07	Book available ticket	Booking created with correct status	Fill after execution
TC-08	Request more tickets than available	Request rejected without overselling	Fill after execution
TC-09	Payment success callback	Payment state updated once	Fill after execution
TC-10	Repeated payment callback	No duplicate ticket or charge record	Fill after execution
TC-11	Generate ticket QR	QR corresponds to stored ticket token	Fill after execution
TC-12	Scan valid unused ticket	Entry accepted and ticket marked used	Fill after execution
TC-13	Scan ticket a second time	Duplicate entry rejected	Fill after execution
TC-14	Scan invalid token	Entry rejected	Fill after execution
TC-15	Request recommendations	Eligible events returned or fallback shown	Fill after execution
TC-16	View admin report	Counts match underlying records	Fill after execution

5.3 Integration and Security Testing

Integration tests should verify that a successful booking is represented consistently across the booking, ticket and payment records. The team should also test interrupted flows: a payment failure, a network timeout, an expired session, an event becoming full during checkout and a QR scan while the backend is unavailable. Each case should have a documented recovery behaviour.

Security tests should confirm that one user cannot access another user's ticket by changing an identifier, non-admin users cannot create or delete events, expired or malformed tokens are rejected, and secrets are not exposed in client-side bundles or logs. The system should avoid returning stack traces or database details in public error messages.

5.4 Performance Analysis

Review-4 shows illustrative figures of approximately 350 ms average response time, 30 test cases, 29 successful cases, a 3.3% error rate, 45 ms database response and 180 ms API response. The same review explicitly instructs the team to replace these sample values with measurements from its own test runs. They are therefore not presented in this report as verified results.

Metric	How to measure	Recorded result
Total test cases	Count executed test cases	Enter measured value
Passed tests	Count passed tests with evidence	Enter measured value
Pass rate	Passed / executed × 100	Calculate from actual counts
Average response time	Measure repeated API calls under stated conditions	Enter measured value
Median / p95 response time	Collect repeated request durations	Enter measured value
Database response time	Measure database operation duration	Enter measured value
Error rate	Failed requests / total requests × 100	Enter measured value
Test load	Record concurrent users or request rate	Enter measured setup

5.5 Results and Discussion

The final results section should report what the current build actually demonstrates. At minimum, the team should attach screenshots for the login page, event listing, booking form, admin event form, dashboard and QR e-ticket if these screens exist. For each function, state the test input, expected output, actual output and whether the result passed. Screenshots should be readable and should not expose passwords, private tokens or real payment details.

If all core modules are connected, discuss the end-to-end sequence rather than reporting each module in isolation. For example, show how an event is created, how a user books a ticket, how a payment status is recorded, how the QR is generated and how a valid or duplicate scan is handled. AI results should include the dataset description, baseline comparison and evaluation metric.



If an AI feature is only a design or mock response, label it as such.

5.6 Interpretation of AI Results

A recommendation result should be reviewed for relevance and availability: suggested events must not be cancelled, past or sold out. A demand estimate should be compared with actual attendance after the event, not just with registration counts at the time of prediction. A fraud/anomaly flag should be assessed against labelled examples if such labels exist. Small synthetic datasets are useful for testing data flow but are not sufficient evidence of general predictive performance.

The report should avoid using a single accuracy figure as the only measure. Class imbalance, false positives, false negatives and the cost of an incorrect decision matter. A simple baseline is an important comparison because a complex model is not useful if it does not improve on a transparent rule or historical average.

5.7 Limitations

- AI recommendation quality depends on the availability and quality of preference or interaction data.
- Demand forecasting requires historical events and a clear definition of the prediction target.
- Payment integration may be demonstrated in sandbox or mock mode rather than with live transactions.
- QR scanning requires a working camera/device and network access to validate against current ticket status.
- Performance measured on a laptop does not guarantee the same performance under large public-event traffic.
- Fraud detection can produce false positives and should be paired with clear review procedures.
- Review-4 includes sample test and performance values that must be replaced by actual measured evidence.
- Alternative technology stacks appear across review material; the final report must be aligned with the actual codebase.

6. CONCLUSION AND FUTURE WORK

This project proposes an integrated AI-enabled event registration and ticketing platform that connects event discovery, user accounts, booking, payment-status tracking, QR e-tickets and organiser reporting. The review material identifies practical limitations in fragmented event-management workflows and proposes a centralized design with consistent relationships among users, events, registrations, tickets and payments. The Review-4 stack specifies React.js, Node.js/Express, MongoDB and a Python scikit-learn AI service.

The key design principle is to make the transactional workflow reliable before relying on AI. Correct capacity checks, clear booking states, secure payment-status handling and single-use ticket validation are essential foundations. Recommendation and demand-forecasting features can then provide additional value if the team has suitable data and evaluates the models carefully.

The final claim that objectives have been achieved must be based on the implementation and test evidence collected by the team. Before submission, all illustrative performance figures should be replaced, actual screenshots should be inserted, and references should be completed with the academic sources genuinely consulted.

6.1 Future Work

- Add seat maps, waitlists and automated waitlist promotion when capacity becomes available.
- Develop a mobile application with push notifications and offline-friendly ticket access.
- Improve demand forecasting using more historical events and event-specific evaluation.
- Add dynamic pricing only with transparent rules and organiser controls.
- Explore stronger ticket authenticity using signed QR tokens and audit trails.
- Evaluate face-recognition entry only after careful privacy, consent, fairness and legal review; QR validation remains the baseline.
- Add multilingual interfaces and accessibility improvements.
- Introduce monitoring, backups, role-based audit logs and load testing for larger deployments.
- Evaluate AI recommendation quality through documented user feedback and controlled tests.

REFERENCES

Note: The following are foundational technology documentation references suitable for completing the implementation bibliography. Add the specific academic papers actually reviewed through Google Scholar, IEEE Xplore or ACM Digital Library. Do not cite a paper as read unless the team has accessed and reviewed it.

[1] React Team, “React Documentation.” Available: https://react.dev/

[2] OpenJS Foundation, “Node.js Documentation.” Available: https://nodejs.org/docs/

[3] Express.js, “Express – Node.js web application framework.” Available: https://expressjs.com/

[4] MongoDB, “MongoDB Manual.” Available: https://www.mongodb.com/docs/

[5] scikit-learn developers, “scikit-learn User Guide.” Available: https://scikit-learn.org/stable/user_guide.html

[6] Postman, “Postman Learning Center.” Available: https://learning.postman.com/

[7] OWASP Foundation, “OWASP Cheat Sheet Series.” Available: https://cheatsheetseries.owasp.org/

[8] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG).” Available: https://www.w3.org/WAI/standards-guidelines/wcag/

[9] Payment provider documentation selected by the team (add exact provider, API version and access date).

[10] QR-generation/scanning library documentation selected by the team (add exact package and version).

[11] Academic papers actually consulted for event management, ticketing, QR verification, recommender systems and attendance forecasting (complete author, title, venue, year and DOI/URL).

APPENDIX A: MAPPING OF FOUR PBL REVIEWS TO THE REPORT

Review material	Core requirements in source	Report location
Review-1	Title, problem statement, societal need, objectives, proposed solution/workflow, feasibility and novelty	Chapter 1; Sections 2.1 and 2.5
Literature Review / Review-2 material	Existing approaches, research approach, comparative study, gap analysis, admin/user modules and database design	Sections 2.2–2.9
Review-3	Problem identification, data collection/preprocessing, architecture, algorithms, workflow, implementation, testing and deployment	Chapters 2–5
Review-4	Objectives, development approach, React/Node/MongoDB/Python stack, architecture, screens, functional results, testing and sample performance	Sections 1.4, 2.7, Chapter 4 and Chapter 5

The Review-1 specifies a seven-slide structure and calls for a problem statement, societal need, objectives, proposed solution/methodology, feasibility and novelty. The supplied Review-3 describes the proposed AI/data pipeline and implementation/testing plan. The literature-review presentation identifies a centralized integration gap. Review-4 includes sample metrics and expressly says they must be replaced with measured results.

APPENDIX B: EVIDENCE TO ATTACH BEFORE FINAL SUBMISSION

37. Insert the university logo and confirm the report cover page format required by the faculty.
38. Insert the actual screenshots for login, event list, booking, add/update event, dashboard and QR ticket.
39. Attach a screenshot or export of the Postman test collection and actual responses.
40. Replace the sample testing table with real outcomes, including failed tests and fixes.
41. Run repeated response-time measurements and record the hardware, network, request count and test conditions.
42. Document the final chosen stack and align every technology statement with the source code.
43. Include the actual database schema or representative sample documents with private information removed.
44. Describe the AI dataset source, record count, fields, labels, preprocessing and train/test split.
45. Include model outputs and evaluation metrics only if models were trained and evaluated.
46. Complete the bibliography with the papers and API/library documentation actually consulted.
47. Check team member names, roll numbers, mentor name, department and course code against official records.
48. Remove any template instructions, placeholders or unverified claims before submission.

APPENDIX C: SAMPLE DATA DICTIONARY

Field	Type (example)	Validation / notes
userId	ObjectId/string	Unique stable identifier
email	String	Required, normalised, unique
role	Enum/string	Attendee, organiser/admin according to access policy
eventId	ObjectId/string	Must refer to an existing event
capacity	Integer	Non-negative; cannot be lower than already committed tickets without a policy
availableCount	Integer	Derived/maintained safely; must not become negative
bookingStatus	Enum/string	Pending, confirmed, cancelled, failed, as implemented
paymentStatus	Enum/string	Pending, succeeded, failed, refunded, as implemented
ticketToken	Random/signed token	Unique; do not encode personal data
ticketStatus	Enum/string	Issued, used, revoked, or equivalent documented states
scannedAt	Timestamp	Stored consistently, preferably UTC
recommendationScore	Number	Optional score; explain its interpretation
predictedAttendance	Number	Forecast estimate, not an observed count
actualAttendance	Integer	Count derived from verified check-ins where possible

This data dictionary is illustrative. The final field names and data types must match the actual database implementation.

APPENDIX D: SAMPLE TEST EXECUTION RECORD

Use one record per test execution. Preserve failed results rather than overwriting them after a fix; a second execution can be recorded separately.

Field	Value to record
Test ID	Unique identifier from the test plan
Build/version	Git commit, version number or build date
Date and tester	Actual execution date and team member
Preconditions	Account role, event capacity, payment mode and relevant setup
Input/action	Exact action or API request
Expected result	Requirement-based expected behaviour
Actual result	Observed response or screen behaviour
Status	Pass / Fail / Blocked
Evidence	Screenshot, response export or log reference
Defect/fix	Issue, severity and follow-up execution if applicable

APPENDIX E: GLOSSARY

Term	Meaning in this project
AI	Artificial Intelligence; here, recommendation and forecasting support
API	Application Programming Interface used by frontend/backend services
Booking	A reservation record connecting a user and event
Demand forecast	Estimate of future registrations or attendance
E-ticket	Electronic ticket associated with a booking
JWT	JSON Web Token, one possible mechanism for representing an authenticated session
MongoDB	Document-oriented database selected in the Review-4 target stack
QR code	Two-dimensional code used to carry a ticket token or verification reference
REST	API design style commonly used for resource-oriented HTTP endpoints
Scikit-learn	Python machine-learning library named for the AI module
No-show	A registered participant who does not attend the event
Idempotency	Handling repeated requests so they do not create duplicate side effects

APPENDIX F: USER STORIES AND ACCEPTANCE CRITERIA

User stories express the system from the perspective of the people who use it. They help the team confirm that a feature has a clear purpose and a testable outcome. The stories below are proposed acceptance criteria and should be checked against the final implementation.

ID	User story	Acceptance criteria
US-01	As an attendee, I want to browse events so I can decide which event to attend.	The event list shows active events and each event opens a details page.
US-02	As an attendee, I want to register and log in so my bookings are associated with my account.	Valid credentials work; invalid input is rejected with a clear message.
US-03	As an attendee, I want to reserve a ticket so I can secure a place.	The server checks availability and returns a clear booking status.
US-04	As an attendee, I want to view my e-ticket so I can enter the event.	A confirmed booking displays a ticket token/QR linked to a stored ticket.
US-05	As entry staff, I want to scan a ticket so I can validate entry.	The system returns accepted or rejected with a reason and records the scan.
US-06	As an organiser, I want to manage event information so the catalogue stays accurate.	Only authorised roles can create, update or cancel events.
US-07	As an organiser, I want to view attendance information so I can plan resources.	Dashboard counts are based on stored booking and check-in records.
US-08	As an attendee, I want relevant event suggestions so I can discover events.	Recommendations are drawn from available events and preferences or a labelled fallback.

Acceptance criteria should be demonstrated with a reproducible test. A feature should not be considered complete merely because a page exists; its data, validation, access permissions and failure cases must also work.

APPENDIX G: ERROR HANDLING AND RECOVERY PLAN

A reliable ticketing system must define what happens when a request fails or a user repeats an action. Clear error handling protects both the attendee and the organiser from inconsistent records. The following cases should be included in development and testing.

Situation	Expected system behaviour	Recovery / evidence
Event is full at booking time	Reject new reservation and show unavailable status.	Refresh event availability; retain no confirmed over-capacity booking.
Payment provider times out	Keep booking in a pending state until verified or expired.	Allow safe status refresh; do not issue duplicate tickets.
Payment callback arrives twice	Process the same provider reference idempotently.	Confirm only one payment transition and one ticket issuance.
QR scanner has no network	Do not falsely claim server validation succeeded.	Show that online validation is unavailable and follow venue fallback policy.
Ticket already used	Reject a second entry attempt.	Display already-used status and retain scan audit record.
User session expires	Reject protected request without exposing private data.	Ask user to sign in again and retry safely.
Database unavailable	Return a controlled service error.	Log technical details securely and avoid issuing unconfirmed tickets.
Notification fails	Keep the valid booking accessible in the user account.	Record delivery failure and offer retry or alternate retrieval.

For each error path, the team should record the observed response, database state before and after the action, and whether recovery was successful. This is especially important for payment, booking and ticket validation because these operations can affect capacity and event entry.

APPENDIX H: DEMONSTRATION SCRIPT FOR PROJECT REVIEW

The demonstration should follow one complete scenario rather than showing disconnected pages. Use test accounts and sandbox/mock payment data. Do not display real passwords, secret keys, personal attendee details or live payment information.

1. Sign in as an authorised organiser and create a sample event with a date, venue, ticket capacity and ticket price.

2. Open the attendee view and confirm that the event appears in the catalogue with correct information.

3. Register or sign in as a test attendee and open the event details page.

4. Select a valid ticket quantity and submit a booking; show the response and resulting booking status.

5. Complete the sandbox/mock payment path and demonstrate that payment status is recorded as expected.

6. Open the attendee ticket page and show the generated QR code and non-sensitive booking reference.

7. Scan the ticket with the intended device or use the validation endpoint in Postman; show the accepted response.

8. Scan the same ticket again and demonstrate that duplicate entry is rejected if single-use validation is implemented.

9. Return to the organiser dashboard and compare booking and attendance counts with the test records.

10. Open the recommendation or demand-forecast view only if the AI feature is connected; explain the input data and show the actual output.

11. Run one negative test, such as an invalid QR token, unauthorised admin request or booking beyond capacity.

12. Finish by stating the current limitations and which future features remain planned.

The team should capture screenshots during the real demonstration and replace any placeholder screens in the review slides or final report. If a step is not implemented, state that clearly instead of presenting a static mock-up as a working feature.